

# Assignment (19.3)

---

Name: Manupati Eshwar

Hall Ticket No: 2303A51495

## Task 1 – Python to C++ Conversion

Prompt:

Convert a Python Student class into C++.

Python Code:

```
class Student:
```

```
    def __init__(self, name, marks):
```

```
        self.name = name
```

```
        self.marks = marks
```

```
    def display(self):
```

```
        print(self.name, self.marks)
```

C++ Code:

```
#include <iostream>
```

```
using namespace std;
```

```
class Student {
```

```
public:
```

```
    string name;
```

```
    int marks;
```

```
    Student(string n, int m){
```

```
        name = n; marks = m;
```

```
    }
```

```
    void display(){
```

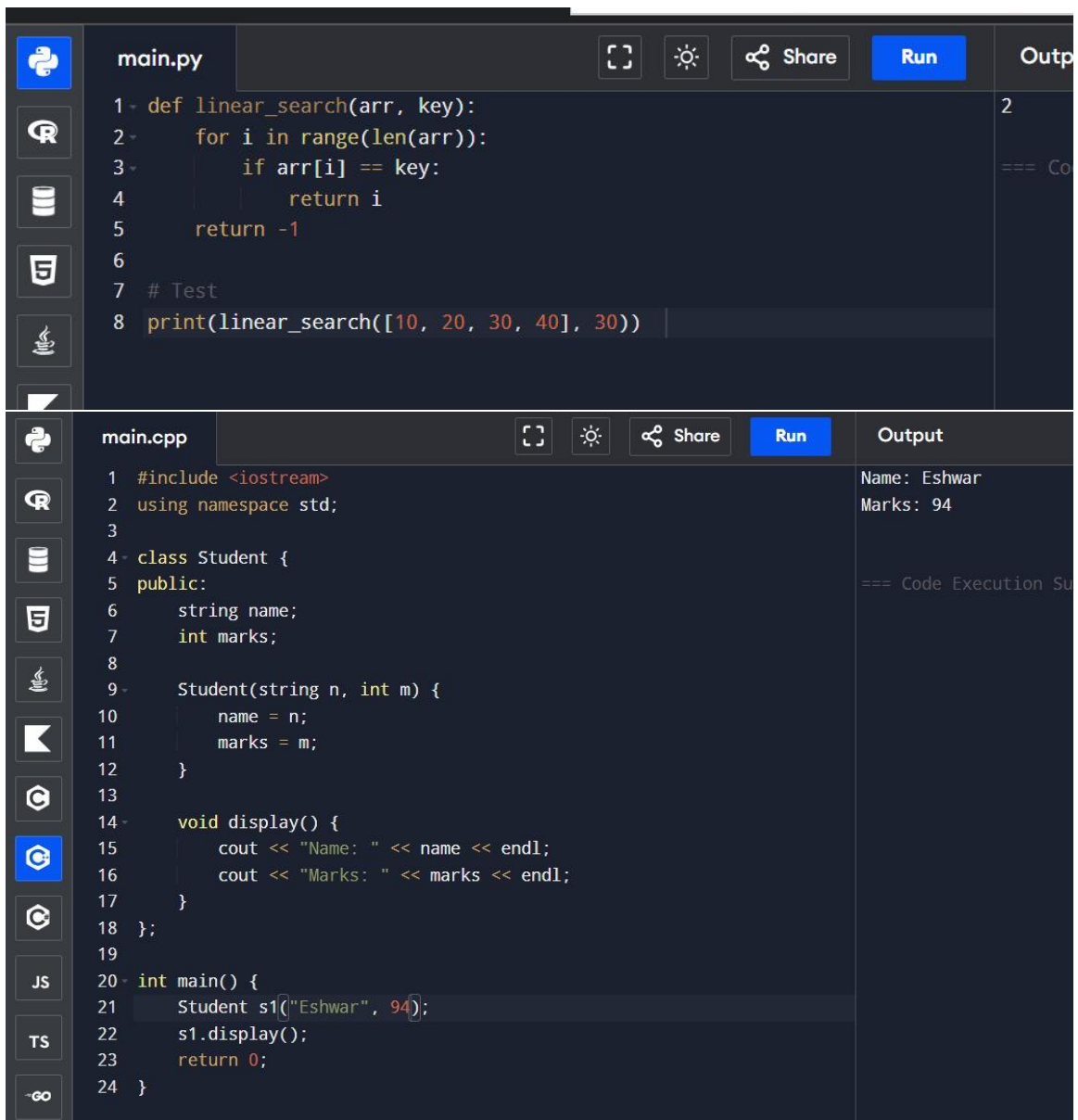
```
        cout << name << " " << marks;
```

```
    }
```

```
};
```

Output:

Same output in both languages



## Task 2 – Java to Python Conversion

Prompt:

Convert Java `isPrime()` method into Python.

Java Code:

```
boolean isPrime(int n){
    if(n<=1) return false;
    for(int i=2;i<n;i++){
        if(n%i==0) return false;
```

```

    }
    return true;
}

```

Python Code:

```

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

```

Output:

Input: 7 → True

Input: 4 → False

The screenshot shows a code editor with two tabs: 'main.py' and 'Main.java'. Both tabs have a 'Run' button and an 'Output' pane on the right. The Python code in 'main.py' defines a function 'is\_prime(n)' that returns True for prime numbers and False otherwise. It then prints the results for 7 and 4. The Java code in 'Main.java' defines a class 'Main' with a static method 'isPrime(int n)' that returns true for prime numbers and false otherwise. It then prints the results for 7 and 4. The output panes show the results of the code execution: 'True' and 'False' for Python, and 'true' and 'false' for Java.

| File      | Code                                                                                                                                                                                                                                                                                                                                                                                                                                            | Output                  |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| main.py   | <pre> 1- def is_prime(n): 2-     if n &lt;= 1: 3-         return False 4-     for i in range(2, n): 5-         if n % i == 0: 6-             return False 7-     return True 8- print(is_prime(7)) 9- print(is_prime(4)) </pre>                                                                                                                                                                                                                 | <pre> True False </pre> |
| Main.java | <pre> 1- public class Main { 2-     public static boolean isPrime(int n) { 3-         if (n &lt;= 1) return false; 4- 5-         for (int i = 2; i &lt; n; i++) { 6-             if (n % i == 0) 7-                 return false; 8-         } 9-         return true; 10-    } 11- 12-    public static void main(String[] args) { 13-        System.out.println(isPrime(7)); 14-        System.out.println(isPrime(4)); 15-    } 16- } </pre> | <pre> true false </pre> |

### Task 3 – Pseudocode to Python

Prompt:

Convert bubble sort pseudocode to Python.

Python Code:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

```
print(bubble_sort([5,2,9,1]))
```

Output:

[1, 2, 5, 9]

A screenshot of a code editor interface. The editor has a dark theme. On the left, there is a sidebar with icons for Python, a file explorer, a search bar, and a terminal. The main area shows a file named 'main.py' with the following code:

```
1- def bubble_sort(arr):
2-     n = len(arr)
3-     for i in range(n):
4-         for j in range(0, n-i-1):
5-             if arr[j] > arr[j+1]:
6-                 arr[j], arr[j+1] = arr[j+1], arr[j]
7-     return arr
8
9 # Test
10 print(bubble_sort([5, 2, 9, 1]))
```

On the right side of the editor, there is a 'Run' button and an 'Output' panel. The 'Output' panel shows the result of the code execution: '[1, 2, 5, 9]'. Below the output, it says '=== Code Executed ==='.

### Task 4 – SQL to Pandas

Prompt:

Convert SQL query to Pandas.

SQL:

```
SELECT name, salary FROM employees WHERE salary > 50000
```

Python Code:

```
import pandas as pd
```

```
data = {"name":["A","B","C"],"salary":[40000,60000,70000]}
df = pd.DataFrame(data)
result = df[df["salary"] > 50000][["name","salary"]]
print(result)
```

Output:

B 60000

C 70000

The screenshot shows a Jupyter Notebook interface with a dark theme. On the left is a sidebar with icons for file explorer, search, and other tools. The main area is split into two panes. The left pane, titled 'main.py', contains the following Python code:

```
1 import pandas as pd
2
3 data = {
4     "name": ["A", "B", "C"],
5     "salary": [40000, 60000, 70000]
6 }
7
8 df = pd.DataFrame(data)
9
10 result = df[df["salary"] > 50000][["name", "salary"]]
11
12 print(result)
```

The right pane, titled 'Output', displays the result of the code execution as a table:

|   | name | salary |
|---|------|--------|
| 1 | B    | 60000  |
| 2 | C    | 70000  |

Below the table, the text '=== Code Execution Success' is visible.

## Task 5 – Algorithm Translation Across Languages

Prompt:

Convert Python linear search to C.

Python Code:

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
```

C Code:

```
#include <stdio.h>

int linearSearch(int arr[], int n, int key){
    for(int i=0;i<n;i++){
        if(arr[i]==key) return i;
    }
    return -1;
}
```

}

Challenges:

- Syntax differences
- Memory management
- Library support

The image displays two side-by-side code editors, each showing a different implementation of a linear search function. The top editor, titled 'main.py', contains Python code. The bottom editor, titled 'main.c', contains C code. Both editors have a dark theme and include icons for file operations, a 'Run' button, and a 'Share' button. The Python code defines a function 'linear\_search' that iterates through an array and returns the index of the first element matching the key, or -1 if not found. The C code defines a function 'linearSearch' with similar logic, using 'printf' to output the result in the main function. The output of the Python code is '2' and the output of the C code is 'Index: 2'.

```
main.py
1- def linear_search(arr, key):
2-     for i in range(len(arr)):
3-         if arr[i] == key:
4-             return i
5-     return -1
6
7 # Test
8 print(linear_search([10, 20, 30, 40], 30))
```

```
main.c
1 #include <stdio.h>
2
3 int linearSearch(int arr[], int n, int key) {
4     for(int i = 0; i < n; i++) {
5         if(arr[i] == key)
6             return i;
7     }
8     return -1;
9 }
10
11 int main() {
12     int arr[] = {10, 20, 30, 40};
13     int result = linearSearch(arr, 4, 30);
14
15     printf("Index: %d", result);
16     return 0;
17 }
```